Ders 11 Genişletilmiş Notu: CFG'den PDA'ya Dönüştürme, Dil Hiyerarşisi ve Turing Makinelerine Giriş

Ders 11 Genişletilmiş Notu: CFG'den PDA'ya Dönüştürme, Dil Hiyerarşisi ve Turing Makinelerine Giriş

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Yığın Otomatlarının Tekrarı
- 2. CFG'den PDA'ya Dönüştürme
- 3. Örnek: $S \to aSTb \mid b$, $T \to Ta \mid \varepsilon$
- 4. Üretilen PDA En Soldan Türetimi Taklit Eder
- 5. Teorem ve Ters Yön: PDA'dan CFG'ye
- 6. Kitapta Neredeyiz?
- 7. Dil Hiyerarşisi
- 8. Turing Makinesinin Yapısı
- 9. Başlangıç Şeridi ve Boşluk Sembolü
- 10. Geçişlerin Gösterimi
- 11. Turing Makinesinde Anlık Betimleme
- 12. Turing Makineleri Deterministiktir
- 13. Kısmi Geçiş Fonksiyonu
- 14. Durma ve Kabul Durumları
- 15. Kabul, Ret ve Sonsuz Döngü
- 16. Örnek: a^* Tanıyan Makine
- 17. Hiç Taramadan Kabul Eden Makine
- 18. Örnek: Sonsuz Döngüye Giren Makine
- 19. Durma Problemi
- 20. Evde İzlenecek Örnek: $\{a^n b^n\}$ Makinesi

1. Yığın Otomatlarının Tekrarı

Geçen hafta yığın otomatlarına (pushdown automata, PDA) başlamıştık. Bir PDA'da alıştığımız giriş dizgisi ve durumların yanında bir de yığın (stack) vardır. Hoca derse bu makinelerin temel kavramlarını hatırlatarak başladı.

1.1 Anlık Betimleme

PDA'nın anlık betimlemesi (instantaneous description, ID) üç elemanlı bir demettir:

- Birinci eleman, makinenin bulunduğu durumdur. Örneğin q_1.
- İkinci eleman, girişin henüz işlenmemiş kısmıdır.
- Üçüncü eleman, yığının o andaki içeriğidir.

Bunun pratik bir faydası vardır: Makinenin bütün resmini her adımda yeniden çizmek yerine yalnız ID'yi yazarsınız. Durum diyagramı elinizde bir yerde durduğu sürece, ona bakarak ID'leri art arda yazmaya devam edebilirsiniz.

1.2 Deterministik Olmama ve Hesaplama Ağacı

PDA'lar deterministik değildir (non-deterministic). Bu yüzden bazı noktalarda hesaplama çoğalır ve bir ağaca dönüşür. Hoca bunu şöyle anlattı: Hesaplama o noktada kopyalanır, artık paralel çalışan iki ya da daha fazla makine vardır.

Bu hesaplama ağaçlarını (computation tree) yazabilirsiniz, ama yazmak zorunda değilsiniz. Bazen hangi hesaplamanın işe yarayacağını görebilirsiniz. Görebiliyorsanız doğrudan onu yazın. Göremiyorsanız neler olduğunu anlamak için ağacı yazmak isteyebilirsiniz.

1.3 Geçişlerin Okunuşu ve Dizgi Kısaltması

PDA'daki bir geçiş üç şey söyler: girişten okunan ilk sembol, yığından çekilen (pop) sembol ve yığına itilen (push) sembol.

Normalde tek sembol çekilir ve tek sembol itilir. Ama geçişi, bir dizgi çekiliyor ya da itiliyormuş gibi de yazabilirsiniz. Bu yalnızca bir kısaltmadır. Örneğin $eb \to cdf$ şu anlama gelir: önce e çekilir, sonra b çekilir, sonra f, ardından d, en son c itilir. Hoca, bu adımların arasında başka hesaplamaların da olabileceğini ekledi.

1.4 Biçimsel Tanım

PDA biçimsel olarak yedili bir demettir (7-tuple). Birinci eleman durumlar kümesidir. İkinci eleman Σ, girişte kullanılan sembollerin kümesidir. Üçüncü eleman Γ, yığında kullanabileceğimiz sembollerin kümesidir. Dördüncü eleman geçiş fonksiyonudur (transition function).

Geçiş fonksiyonunun değerleri birer kümedir. Sebebi yine deterministik olmamadır: Aynı giriş ve aynı yığın tepesi için gidilebilecek birden fazla durum olabilir. Hatta durum aynı kalıp yığına itilen şey farklı olabilir. Hoca bunu aynı q_2 durumuna farklı iki dizgi iten elemanlarla gösterdi.

Hoca Γ için ayrıca, bunun özel yığın sembolünü içermediğini söyledi.

> **Ders sonrası not:** Hoca Γ'nın özel yığın sembolünü içermediğini söyledi. Bu, $\$$ yığına hiç yazılmaz demek değildir: Derste $\$$ yığında duruyor ve $\$ \to \$$ geçişiyle çekilip geri itiliyor. Bu yüzden $\$$'ı Γ dışında tutan bir tanımda da geçiş fonksiyonunun onu yığın tepesi olarak görebilmesi gerekir.

1.5 Hesaplama ve PDA'nın Dili

Art arda gelen ID'lerin listesine hesaplama (computation) denir. İsterseniz kısa bir gösterimle yazabilirsiniz. Bu gösterimin slayttaki örneğini hoca şöyle okudu: Başlangıç durumunda, girişin tamamı duruyor ve yığın boş. Burada özel yığın sembolünü saymıyoruz. Bir miktar hesaplamadan sonra q_3'e varıyoruz, bütün giriş kullanılmış ve yığın başladığı hale dönmüş.

Bir PDA'nın dili (language), o makinenin kabul ettiği bütün dizgilerin kümesidir. Yani bu dizgilerin her biri için şu mümkündür: Başlangıç durumunda, o dizgiyle ve yığın başlangıç sembolüyle başlanır. Bir miktar hesaplamadan sonra bir kabul durumuna gidilir ve giriş tamamen tüketilmiş olur. Yığında bir şey kalmış olabilir. Ne kaldığının önemi yoktur.

> **Hocanın vurgusu:** Kabul durumuna varmak çok önemlidir. Girişin tamamı da tüketilmiş olmalıdır. Yığında ne kaldığı ise önemli değildir.

Hoca örnek olarak geçen haftaki M_1 makinesini hatırlattı: $L(M_1) = \{a^n b^n \mid n \geq 0\}$.

1.6 Eşdeğerlik

Bağlamdan bağımsız diller (context-free languages, CFL) ile PDA'lar eşdeğerdir. Bağlamdan bağımsız bir gramerin (context-free grammar, CFG) ürettiği dili bir PDA kabul eder. Bir PDA'nın kabul ettiği dili de bir CFG üretir. Geçen hafta bunu ifade etmiş ve dönüşümlerin nasıl yapıldığını görmemiz gerektiğini söylemiştik. Bu dersin ilk kısmı bu dönüşüme ayrıldı.

2. CFG'den PDA'ya Dönüştürme

2.1 İspatın İki Yönü

Böyle bir eşdeğerliği ispatlarken aslında iki şey ispatlamamız gerekir. Bunlar küme olduğu için, iki kümenin eşitliğini göstermek demek birinci kümenin ikincinin alt kümesi olduğunu ve üst kümesi olduğunu ayrı ayrı göstermek demektir.

Önce birinci yöne bakıyoruz: CFL'ler PDA dillerinin alt kümesidir. Bunun anlamı, her CFG'nin bir PDA'ya dönüştürülebilmesidir. Yani keyfi bir G grameri alacağız ve onu $L(G) = L(M)$ olan bir M makinesine çevireceğiz.

2.2 Üç Durumlu Yapı

Bu yöndeki dönüştürme işlemi aslında oldukça kolaydır. Yeni PDA'da yalnız üç durum vardır. Her zamanki gibi q_0 ile başlanır, q_0'dan q_1'e, q_1'den q_2'ye geçilir. q_2 kabul durumudur.

Bu iki geçiş her gramer için aynıdır:

- $q_0 \to q_1$**:** $\varepsilon, \varepsilon \to S$. Girişten hiçbir şey okunmaz, yığından hiçbir şey çekilmez, S itilir. Burada S, gramerin başlangıç değişkenidir (starting variable).
- $q_1 \to q_2$**:** $\varepsilon, \$ \to \$$. Girişten hiçbir şey okunmaz. Yığında $\$$ varsa çekilir ve geri itilir.

Son geçişle aslında yığında yalnız $\$$ kalıp kalmadığı kontrol edilir. Her zamanki gibi yığında başlangıçta özel yığın sembolü $\$$ vardır. Bu geçiş, yığında yalnız $\$$ kaldığında çalışabilir.

Hoca gramerin ne olduğunu da hatırlattı. CFG, kurallardan oluşan bir listedir: S ile başlarsınız, S bir şey üretir, başka bir değişken bir w dizgisini üretebilir, bazı değişkenler de terminal üretir. Büyük harfli S ve A değişkendir (variable), küçük a terminaldir (terminal).

2.3 q_1'deki Döngü

İşin ilginç kısmı q_1'dir. q_1 üzerinde bir döngü vardır. Aslında çok sayıda döngü olabilir, ama hepsini tek bir döngü üzerinde gösteririz ve üzerine farklı geçişleri yazarız. Bu geçişler iki türlüdür:

- **Her üretim kuralı (production rule)** $A \to w$ **için:** $\varepsilon, A \to w$. Girişten hiçbir şey okunmaz, A değişkeni çekilir, w dizgisi itilir.
- **Her terminal** a **için:** $a, a \to \varepsilon$. Girişten a okunur, yığından a çekilir, hiçbir şey itilmez.

Gramerde $A \to w$ yazısını "A, w'yu üretir" diye okuruz. PDA'da buna karşılık gelen şey bir geçiştir ve "girişten hiçbir şey okuma, değişkeni çek, dizgiyi it" demektir. Hoca önce "her değişken için" dedi, sonra düzelterek bunun her üretim kuralı için olduğunu söyledi.

3. Örnek: $S \to aSTb \mid b$, $T \to Ta \mid \varepsilon$

Hoca, işlemin ne kadar basit olduğunu şu gramerle gösterdi:

$$S \to aSTb, \qquad S \to b, \qquad T \to Ta, \qquad T \to \varepsilon$$

3.1 PDA'nın Kurulması

$q_0 \to q_1$ geçişi S'yi iter. $q_1 \to q_2$ geçişi $\varepsilon, \$ \to \$$ olur ve q_2 kabul durumudur. Yığında başlangıçta yine $\$$ vardır.

q_1'e önce dört üretim kuralı için dört geçiş eklenir. Sonra iki terminal a ve b için iki geçiş eklenir. Böylece q_1'de altı geçiş olur:

Kaynak	q_1'deki geçiş	Anlamı
$S \to aSTb$	$\varepsilon, S \to aSTb$	Okuma yok, S çek, $aSTb$ it
$S \to b$	$\varepsilon, S \to b$	Okuma yok, S çek, b it
$T \to Ta$	$\varepsilon, T \to Ta$	Okuma yok, T çek, Ta it

Kaynak	q_1'deki geçiş	Anlamı
$T \to \varepsilon$	$\varepsilon, T \to \varepsilon$	Okuma yok, T çek, hiçbir şey itme
terminal a	$a, a \to \varepsilon$	a oku, a çek, hiçbir şey itme
terminal b	$b, b \to \varepsilon$	b oku, b çek, hiçbir şey itme

3.2 $abab$ Girişinin Hesaplama Ağacı

Makinenin nasıl çalıştığını görmek için giriş $abab$ alınır ve hesaplama ağacı yazılır. İlk zamanda (time) makine q_0'dadır, giriş $abab$'dir, yığında yalnız \$ vardır.

Zaman 0 ve 1. $(q_0, abab, \$)$ ID'sinden ilk geçiş çalışır ve q_1'e gidilir. Giriş değişmez, ama S itildiği için yığın $S\$$ olur: $(q_1, abab, S\$)$.

Zaman 2. Burada hangi geçişler mümkündür? Yığının tepesinde S var ve S kurallarından türeyen iki geçiş okuma yapmadan S'yi çekebilir. Diğer dört geçiş uymaz. PDA'lar deterministik olmadığı için ikisi de gerçekleşir, yani makine ikiye ayrılır:

- $(q_1, abab, aSTb\$)$: birinci kural uygulandı.
- $(q_1, abab, b\$)$: ikinci kural uygulandı.

Zaman 3. Artık paralel çalışan iki makine var. Birincide yığının tepesi a ve okunacak sembol de a. Bu, $a, a \to \varepsilon$ geçişine uyar ve başka geçiş yoktur. Sonuç $(q_1, bab, STb\$)$ olur. İkinci makinede ise okunacak sembol a, yığının tepesi b'dir. Böyle bir geçiş yoktur. Bu durum kabul durumu da değildir ve giriş de tükenmemiştir. Yani bu makine ölür. Yeniden tek makine kaldı.

Zaman 4. Yığının tepesinde yine S var. Yine iki geçiş uyar ve makine kendini kopyalar:

- $(q_1, bab, aSTbTb\$)$
- $(q_1, bab, bTb\$)$

Zaman 5. Birinci dal ölür: Okunacak sembol b, çekilecek sembol a; böyle bir geçiş yok. İkinci dal işe yarar, çünkü okunacak sembol b ve yığının tepesi de b. Sonuç $(q_1, ab, Tb\$)$ olur.

Zaman 6. Tepede T var ve iki geçiş uyar. Tepede değişken varken girişten okuma yapılmadığı için giriş aynen yazılır:

- $(q_1, ab, Tab\$)$: $T \to Ta$ uygulandı.
- $(q_1, ab, b\$)$: $T \to \varepsilon$ uygulandı.

Zaman 7. İkinci dal ölür: Giriş a, tepe b, böyle bir geçiş yok. Birinci dal yine ikiye ayrılır:

- $(q_1, ab, Taab\$)$: $T \to Ta$ uygulandı.
- $(q_1, ab, ab\$)$: $T \to \varepsilon$ uygulandı.

Hoca bu noktada şunu gösterdi: İkinci dalda giriş ile yığın güzelce eşleşiyor, a ile a, b ile b. Geriye boş giriş ve yalnız \$ kalacak, o da son geçişe uyacak. Bu dal kesinlikle kabul eder. Hoca kalan adımları atladı. Atlanan adımlar şunlardır (ara adım eklendi):

$$(q_1, ab, ab\$) \vdash (q_1, b, b\$) \vdash (q_1, \varepsilon, \$) \vdash (q_2, \varepsilon, \$)$$

q_2 kabul durumu olduğu ve giriş tüketildiği için bu kabul eden bir durumdur.

Kabul eden dalın tamamı şöyledir:

Zaman	ID	Açıklama
0	$(q_0, abab, \$)$	Başlangıç
1	$(q_1, abab, S\$)$	S itildi
2	$(q_1, abab, aSTb\$)$	$S \to aSTb$
3	$(q_1, bab, STb\$)$	a eşleşti
4	$(q_1, bab, bTb\$)$	$S \to b$
5	$(q_1, ab, Tb\$)$	b eşleşti

Zaman	ID	Açıklama
6	$(q_1, ab, Tab\$)$	$T \to Ta$
7	$(q_1, ab, ab\$)$	$T \to \varepsilon$
8	$(q_1, b, b\$)$	a eşleşti
9	$(q_1, \varepsilon, \$)$	b eşleşti
10	$(q_2, \varepsilon, \$)$	Kabul

3.3 Sonsuza Giden Dal

Diğer dal ise hiç bitmez. Tepedeki T, $T \to Ta$ geçişiyle Ta'ya dönüşür ve aynı şey yeni T'ye sonsuza kadar uygulanabilir. Hoca burada bir an durup "aslında sonsuza kadar değil" dedi, sonra kendini düzeltti: Evet, sonsuza kadar sürer. Giriş biter diye düşünmüştü, ama bu dalın girişi okuması gerekmiyor. Yalnız T'yi Ta ile değiştiriyor.

Bu dalın sonsuza kadar çalışması sonucu değiştirmez. Kabul eden bir hesaplama olduğu için dizgi kabul edilir.

3.4 Ağacı Yazmak Zorunda Değilsiniz

Hoca her zamanki uyarısını tekrarladı: Bütün hesaplama ağacını yazmanız gerekmez. $abab$ girişine bakarak çalışacak dalı sezebilirsiniz. Bu örneği daha sonra bütün ağacı yazmadan kendiniz denemenizi istedi.

> **Ders sonrası not:** Örnek gramer ile kurulan PDA, Python ile tam tarandı. Uzunluğu en fazla 9 olan bütün $\{a, b\}$ dizgilerinde (1023 dizgi) PDA ile gramer aynı dili verdi. Zaman zaman yazılan ağaç da yukarıdaki dallarla aynı çıktı.

4. Üretilen PDA En Soldan Türetimi Taklit Eder

İlk bakışta fark edilmeyen bir şey vardır: Üretilen PDA, gramerdeki en soldan türetimi (leftmost derivation) taklit eder. Bunu görmek için yığın içeriklerine bakın. Başta \$ var, sonra $S\$$, sonra $aSTb\$$. Değişkenden önceki terminalden kurtuluyoruz. Sonra en soldaki değişken çekilip değiştiriliyor. Yine terminal çıkıyor ve yine en soldaki değişkene geliyoruz. Yani yığının tepesi dizginin soluna karşılık gelir.

Şu kural elde edilir: Yığın içerikleri en soldan türetimi taklit eder. Tek fark, baştaki terminallerin gizlenmiş ya da silinmiş olmasıdır.

Hoca bunu $abab$ üzerinde, yalnız kabul eden dala bakarak gösterdi. Aynı dizginin gramerdeki türetimi şöyledir:

$$S \Rightarrow aSTb \Rightarrow abTb \Rightarrow abTab \Rightarrow abab$$

Karşılaştırma için yan yana yazalım:

Zaman	Yığın içeriği	Türetimde karşılığı
1	$S\$$	S
2	$aSTb\$$	$aSTb$
3	$STb\$$	$aSTb$ (baştaki a eşleşip silindi)
4	$bTb\$$	$abTb$ (baştaki a silinmiş halde)
5	$Tb\$$	$abTb$ (baştaki ab silinmiş halde)
6	$Tab\$$	$abTab$ (baştaki ab silinmiş halde)
7	$ab\$$	$abab$ (baştaki ab silinmiş halde)
8	$b\$$	$abab$ (baştaki aba silinmiş halde; ara adım eklendi)
9	\$	$abab$ (tamamı silinmiş halde)

Gramer baştaki a'yı tutar ve STb yerine bTb yazar: $abTb$. Yığında ise bu a silinmiştir. Yani dizginin değişkenlerden arınmış baştaki kısmı yığında silinir. Bunun dışında ikisi aynıdır.

5. Teorem ve Ters Yön: PDA'dan CFG'ye

Genel olarak şu gösterilebilir:

$$G,\ w\text{'yu üretir} \iff M,\ w\text{'yu kabul eder}$$

Dolayısıyla $L(G) = L(M)$. Hoca bu genel adımın ayrıntısına girmedi. Bu ispat bir yapıyla ispattır (proof by construction): Her gramer için ona karşılık gelen bir PDA kurabildiğimizi söylüyoruz. Yani gramerleri PDA'ya çevirebiliyoruz. Böylece ispatın birinci kısmı bitmiş olur.

İkinci kısım, yani PDA'dan CFG'ye dönüştürme, karmaşıktır ve derste yapılmadı. Fikri şudur: Keyfi bir PDA alınıp ona karşılık gelen bir G gramerine çevrilebilir. Ama bu durumda ortaya çok büyük bir gramer çıkar. Hoca bu kısmı atladı. İlgilenenler kitaptan okuyabilir.

6. Kitapta Neredeyiz?

Bununla kitabın birinci ve ikinci bölümleri bitti. İkinci bölümde deterministik bağlamdan bağımsız diller (deterministic context-free languages) atlandı ve derste görülmeyecek. Geri kalan kısım işlendi. Şimdi üçüncü bölüme, yani Church-Turing tezine (Church-Turing thesis) başlıyoruz. Turing makinelerini (Turing machine, TM) bu bölümde göreceğiz. Bölüm adındaki teze daha sonra gelinecek.

7. Dil Hiyerarşisi

Hoca önce şimdiye kadar gördüğümüz dil türlerine, yani dil hiyerarşisine (language hierarchy) baktı.

- **Düzenli diller (regular languages):** Öğrendiğimiz en basit dillerdir. Deterministik ya da deterministik olmayan sonlu otomatlar (finite automata) tasarlanabilir. Bu dillerdeki dizgileri sonlu otomatlar kabul eder. Örnek: a^*.
- **Bağlamdan bağımsız diller:** Dizgileri CFG'lerle üretilebilir ya da PDA'larla kabul edilebilir. Örnek: $a^n b^n$.
- **Turing-tanınabilir diller (Turing recognizable languages):** Bugün başladığımız daha güçlü makineler olan Turing makineleri bu dilleri kabul eder. Bunlar bağlamdan bağımsız dillerin ve düzenli dillerin üst kümesidir. Örnek: $a^n b^n c^n$.

Düzenli diller bağlamdan bağımsız dillerin alt kümesidir: $RL \subseteq CFL$. Turing-tanınabilir diller ise ikisinin de üst kümesidir ve çok daha karmaşık olabilir. Örneğin $a^n b^n c^n$ yazılabilir, çünkü bağlamdan bağımsız diller yalnız iki şeyi sayabilir, yalnız iki şeyi eşleştirebilir.

Turing makineleri çok güçlüdür. İleride bilgisayarınızda hesaplayabildiğiniz her şeyin bir Turing makinesiyle hesaplanabildiğini göreceğiz. Hoca bunu "bir tür sanal makine, mantıksal bir makine" diye tarif etti.

> **Ders sonrası not:** Hoca "Bir dilin bağlamdan bağımsız olduğunu söylerseniz otomatik olarak düzenli olduğunu ispatlamış olursunuz; düzenli olmadığını ispatlarsanız bağlamdan bağımsız olmadığını ispatlamış olursunuz" dedi. İkisi de terstir. $RL \subseteq CFL$ olduğu için doğrusu şudur: Bir dil düzenliyse bağlamdan bağımsızdır. Bir dil bağlamdan bağımsız değilse düzenli de değildir. Bunu derste verilen örnek de gösterir: $a^n b^n$ bağlamdan bağımsızdır, ama düzenli değildir.

8. Turing Makinesinin Yapısı

Bir Turing makinesinin üç parçası vardır: şerit, okuma-yazma kafası ve kontrol birimi.

Şerit (tape). Giriş şeridi ya da kısaca şerit denir. Şeritte semboller bulunur. Şerit iki yönde de sonsuzdur: hem sağa hem sola sonsuza kadar devam eder.

Okuma-yazma kafası (read-write head). Her an, bir sonraki okunacak sembolü gösterir. Adındaki "yazma" kelimesine dikkat edin. Turing makinelerinin özel tarafı, şeride geri yazabilmeleridir. Şimdiye kadar PDA'larda ve sonlu otomatlarda girişi yalnız okuyabiliyorduk. Artık geri yazabiliyoruz. Bu, gerçekten bir bilgisayar belleği gibidir: Belleğe yazabilir, bellekten okuyabiliriz. Burada şerit bellek rolündedir.

Hocanın vurgusu: Turing makinesinin farkı şeride yazabilmesidir. Şerit, bilgisayar belleği gibidir.

Kontrol birimi (control unit). Burada yine durumlar vardır: bir başlangıç durumu, başka durumlar, bir ya da daha fazla kabul durumu (accepting state). Aralarında yine geçişler bulunur.

Okuma ve yazmadan sonra kafa sola ya da sağa gider. Örneğin altındaki a'yı okuyabilir, yerine b yazabilir, sonra sola ya da sağa geçebilir. Özetle kafa, her geçişte ya da her zaman adımında (time step) şu üç şeyi yapar:

- Bir sembol okur.
- Bir sembol yazar.
- Sola ya da sağa gider.

9. Başlangıç Şeridi ve Boşluk Sembolü

Şerit iki yönde sonsuz olduğu için başlangıçtaki girişin bir biçimde gösterilmesi gerekir. Giriş nerede, biz neredeyiz? Bu yüzden başlangıç şeridi biraz özeldir.

Önce, girişin başını kafa gösterir. Başlangıçta kafa, girişin ilk sembolünün üzerindedir. Örneğin giriş $abac$ olabilir. Ama bu yeterli değildir: Şerit iki yönde sonsuz olduğuna göre, girişin sağındaki hücrelerin girişe ait olmadığını nasıl anlayacağız?

Bunun için özel bir sembole ihtiyaç vardır: boşluk sembolü (blank symbol). Hoca bunun için $\diamond$ kullandı. $\diamond$ gördüğümüzde orada hiçbir şey yok demektir. Böylece başlangıçta giriş şeride yazılıdır, kafa girişin ilk sembolündedir ve girişin iki yanında yalnız boşluk sembolü vardır:

$$\cdots \diamond \diamond a\ b\ a\ c \diamond \diamond \cdots$$

Kitapta iki fark vardır. Birincisi, boşluk sembolü başka bir işaretle gösterilir ($\sqcup$). İkincisi, kitaptaki makinelerde şeridin yalnız sağ tarafı sonsuzdur.

Hoca iki modelin eşdeğer olduğunu ileride göreceğimizi söyledi. İki tarafta da sonsuz yer olması ile yalnız sağda olması, makinenin kabul ettiği dizgileri değiştirmez. Her iki tarafta sonsuz yer olması Turing makinesine ek güç vermez. Çünkü solda yere ihtiyaç olduğunda her şeyi sağa kaydırabiliriz.

10. Geçişlerin Gösterimi

Bir geçişin üç şeyi söylemesi gerekir: hangi sembol okunur, hangi sembol yazılır, kafa nereye gider. Gösterim şöyledir:

$$q_1 \xrightarrow{a \to b,\ L} q_2$$

Şöyle okunur: a okunur, onun yerine b yazılır ve kafa sola gider. L sol (left), R sağ (right) demektir. Yerine R yazsaydık kafa sağa giderdi. q_1 o anki durum, q_2 gidilecek durumdur.

Bu geçişin gerçekleşmesi için iki koşul gerekir:

- Makine q_1 durumunda olmalıdır.
- Kafa bir a sembolünü göstermelidir.

11. Turing Makinesinde Anlık Betimleme

Turing makinelerinde de PDA'dakine benzer bir anlık betimleme vardır, ama biçimi farklıdır. PDA'larda (q_0, ab, baw) gibi üç elemanlı demetler yazıyorduk. Turing makinesinde ise kafanın gösterdiği yer sürekli değişebilir. Bu yüzden demet yerine şöyle bir yazım kullanılır:

$$abc\ q_0\ de$$

Bunun anlamı şudur: Şeritte sola doğru sonsuza kadar boşluk sembolü var. Sonra abc, sonra de geliyor. Sonra yine sağa doğru hep boşluk var. Kafa d'yi gösteriyor. Makine q_0 durumunda.

Yani durum, şeridi ikiye ayırmak için kullanılır: Solunda kafadan önceki kısım, sağında ise kafanın gösterdiği sembolle başlayan geri kalan kısım durur.

12. Turing Makineleri Deterministiktir

Turing makineleri deterministiktir. Deterministik olmayan Turing makineleri (non-deterministic Turing machine) de vardır, ama onlara açıkça bu ad verilir. Yalnız "Turing makinesi" denirse deterministik makine kastedilir. Deterministik olmayanları daha sonra göreceğiz.

Deterministik olmak ne demektir? Hatırlayın: Deterministik olmayan makinelerde aynı giriş için sonuçta farklı durumlar olabilir. Turing makinesinde bunlar yasaktır:

- **Aynı girdiye iki farklı iş:** q_1'deyken kafanın altında a varken q_2'ye b yazıp sağa gidilebiliyorsa, aynı a için ayrıca q_3'e d yazıp sola gidilemez.
- **Boş geçiş (null transition):** q_1'den q_2'ye hiçbir şey okumadan, bir şey yazıp sola giden bir geçiş olamaz.

13. Kısmi Geçiş Fonksiyonu

Deterministik sonlu otomatlarda (DFA) olası bütün girişler için ne olacağını yazmak zorundaydık. Giriş alfabemiz $\{a, b, c\}$ ise her durum için a'da, b'de ve c'de ne olacağını söylemek gerekiyordu.

Turing makinelerinde bu böyle değildir. Alfabe yine $\{a, b, c\}$ olabilir, ama bir durumda yalnız a için geçiş bulunabilir. b ve c gösterilmeyebilir. Buna izin vardır, makineler deterministik olsa bile. Bu yazıma kısmi geçiş fonksiyonu (partial transition function) denir.

Sonlu otomatlarda bunu yalnız deterministik olmayan sonlu otomatlarda (NFA) serbest bırakmıştık. Turing makinelerinde deterministik olmalarına rağmen serbest bırakıyoruz. Sebebi şudur: Turing makineleri genellikle epey karmaşıktır ve çok kolay büyür. Bu izin, diyagramları biraz basitleştirme çabasıdır.

14. Durma ve Kabul Durumları

Turing makinelerinde durma (halting) kavramı vardır. Makine, izleyebileceği bir geçiş yoksa o durumda durur. Bu iki şekilde olur:

- **Uyan geçiş yoktur:** Durumdan çıkan geçişler vardır, ama onlar örneğin a ister, kafa ise b gösterir.
- **Hiç geçiş yoktur:** Durumdan çıkan hiçbir geçiş yoktur.

Her iki durumda da makine artık çalışmaz. Hoca burada Turing makinelerinin program olarak düşünülebileceğini söyledi: Programınız durmuş olur.

Bununla ilgili bir kural da şudur: Turing makinelerinde kabul durumlarının çıkan geçişleri yoktur. Kabul durumuna gelen geçişler elbette olabilir, ama ondan çıkan geçiş olamaz. Az önceki tanımla birleştirince şu sonuç çıkar: Kabul durumlarında izlenecek geçiş olmadığından makine orada durur. Kabul durumunda olduğu için de kabul eder.

> **Ek bilgi (derste anlatılmadı):** Hocanın anlattığı model kısmi geçiş fonksiyonlu ve birden fazla kabul durumuna izin veren bir modeldir. Bölüm adına göre kitap büyük olasılıkla Sipser'dir; bu kitapta da büyük olasılıkla δ tam bir fonksiyondur ve ayrı birer kabul ve ret durumu vardır. Kısmi δ ve "geçiş yoksa dur" gösterimi büyük olasılıkla Linz'in kitabındakine benzer. İki model aynı dilleri tanır, çünkü tanımsız geçişleri yeni bir ret durumuna yönlendirmek yeterlidir.

15. Kabul, Ret ve Sonsuz Döngü

Özetle üç şey olabilir:

- **Makine kabul durumunda durur:** Giriş dizgisi kabul edilir.

- **Makine kabul olmayan bir durumda durur:** Makine durmuştur ama kabul durumunda değildir. Giriş reddedilir.
- **Makine sonsuz döngüye (infinite loop) girer:** Hiç durmaz, sonsuza kadar devam eder.

  Ders sonrası not: Hoca son iki durumda da giriş dizgisinin reddedildiğini söyledi. Dil açısından sonuç doğrudur: Dizgi kabul edilmez ve dile girmez. Ama sonsuz döngüdeki makine durmadığı için bir ret kararı da vermiş olmaz. Doğru ifade "kabul etmez"dir.

  Ek bilgi (derste anlatılmadı): Turing-tanınabilir bir dilin tanımı bu ayrıma dayanır. Dildeki her dizgiyi makine kabul eder. Dilde olmayan bir dizgide ise makine ya reddeder ya da döngüye girer. Her girişte duran makineye karar verici (decider) denir.

Bir de şuna dikkat edin: Makine, girişin tamamını bitirmeden de kabul durumuna girebilir. Çünkü giriş aslında tüketilmez, yalnız değiştirilir. Bir makine bir dizgiyi, tamamını taramadan kabul edebilir.

16. Örnek: a^* Tanıyan Makine

Hoca çok basit bir makineyle başladı. Giriş alfabesi $\{a, b\}$, başlangıç durumu q_0, kabul durumu q_1:

Durum	Okunan	Yazılan, yön	Sonraki
q_0	a	a, R	q_0
q_0	$\diamond$	$\diamond, L$	q_1 (kabul)

Diyagramda b hiç geçmez. Yani b'yi kabul eden bir geçiş yoktur. Bu makinenin dili a^*'dır. a'lar sürdüğü müddetçe giriş taranır ve kafa sağa gider. Girişin sonunda boşluğa gelince q_1'e geçilir ve kabul edilir.

16.1 aaa Girişi

Başlangıçta şeritte aaa yazılıdır, başka her yer boşluktur, kafa ilk a'dadır ve durum q_0'dır. Diyagramda $\diamond$ yazılmamışsa orası genellikle boşluk demektir.

Zaman	ID	Kullanılan geçiş
0	$q_0\,aaa$	$a \to a, R$
1	$a\,q_0\,aa$	$a \to a, R$
2	$aa\,q_0\,a$	$a \to a, R$
3	$aaa\,q_0\,\diamond$	$\diamond \to \diamond, L$
4	$aa\,q_1\,a$	Çıkan geçiş yok

Zaman 3'te kafa boşluğu gösterir ve durum q_0'dır. Burası döngüden çıkılan yerdir: $\diamond$ okunur, $\diamond$ geri yazılır, kafa sola gider ve q_1'e geçilir. Zaman 4'te makine durur, çünkü q_1'den çıkan geçiş yoktur. q_1 kabul durumu olduğu için aaa kabul edilir.

> **Ders sonrası not:** Hoca zaman 2'deki adımı anlatırken "yaz ve sola git" dedi. Doğrusu sağa gitmektir, çünkü geçiş $a \to a, R$'dir. Hoca zaman 0'daki geçişten sonraki durumu da bir an "zaman 0" diye adlandırdı; o zaman 1'dir.

16.2 aba Girişi

Hoca bu kez şekil çizmeden, ID'lerle ilerledi:

$$q_0\,aba \vdash a\,q_0\,ba$$

Burada q_0'da ve girişte b vardır. Bu durumda b için geçiş yoktur. Makine durur ve reddeder. Neden durdu? q_0'dan b'yi kabul eden bir geçiş olmadığı için. Neden reddetti? q_0 kabul durumu olmadığı için.

17. Hiç Taramadan Kabul Eden Makine

Sonraki makine daha da basittir: Başlangıç durumu q_0 aynı zamanda kabul durumudur. Bu makine her şeyi kabul eder. Hiç tarama yapmadan durur ve kabul eder. Çünkü daha başlangıçta kabul durumundayız. Giriş ne olursa olsun önemi yoktur.

> **Ders sonrası not:** Hoca bu makineye geçerken önce "aynı dili kabul eden daha basit bir makine" demeye başladı. Bu makine a^*'ı değil, bütün dizgileri kabul eder; hoca da bir sonraki cümlede bunu söyledi. Uzunluğu en fazla 10 olan bütün $\{a, b\}$ dizgilerinde (2047 dizgi) Python ile doğrulandı.

18. Örnek: Sonsuz Döngüye Giren Makine

Turing makinelerinde sonsuz döngü olabilir. Hocanın örneğinde yine iki durum vardır, q_0 ve q_1. q_1 kabul durumudur:

Durum	Okunan	Yazılan, yön	Sonraki
q_0	a	a, R	q_0
q_0	b	b, L	q_0
q_0	$\diamond$	$\diamond, L$	q_1 (kabul)

b görünce sola, a görünce sağa gidilir. Girişi aba alalım:

Zaman	ID	Kullanılan geçiş
0	$q_0\,aba$	$a \to a, R$
1	$a\,q_0\,ba$	$b \to b, L$
2	$q_0\,aba$	$a \to a, R$
3	$a\,q_0\,ba$	$b \to b, L$
4	$q_0\,aba$	Aynen devam eder

Zaman 2, zaman 0'ın aynısıdır. Sonsuza kadar böyle gider. Bu sonsuz döngü yüzünden makine hiçbir zaman kabul durumuna ulaşmaz, yani hiç durmaz. Hoca bu durumda giriş dizgisinin reddedildiğini söyledi.

> **Ders sonrası not:** Hoca önce sözlü olarak zaman 2'nin zaman 0'ın aynısı olduğunu doğru söyledi. Ardından "zaman 2, zaman 1'in aynısıdır" dedi. Doğrusu zaman 0'dır. Reddetme ifadesi için 15. bölümdeki nota bakın: Döngüdeki makine kabul etmez, ama durup reddetmiş de olmaz.

> **Ders sonrası not:** Bu makine her dizgide döngüye girmez. Python ile tam taramada, uzunluğu en fazla 10 olan 2047 dizginin 1034'ünü kabul etti, 1013'ünde döngüye girdi. Kabul ettiği dizgiler a^* ile b ile başlayan dizgilerdir. En az bir a'dan sonra b gelen dizgilerde döngüye girer.

19. Durma Problemi

Bir makinenin döngüde olduğunu nasıl anlarız? Bu kolay görünmez. Hesaplamaları takip edebiliriz. Geçmişimizde zaten bulunan bir ID'ye geri dönersek döngüdeyiz demektir ve hiçbir zaman kabul durumuna ulaşmayacağız. Ama bu da kolay değildir. Döngü çok, çok uzun bir hesaplamadan sonra başlayabilir ve onu bekleyecek kadar zamanımız olmayabilir.

> **Ek bilgi (derste anlatılmadı):** Makine durmadan da ID tekrar etmeyebilir. Örneğin her adımda boşluğun üzerine bir sembol yazıp sağa giden bir makinede şerit içeriği hep uzar ve ID hep yenidir. Bu yüzden ID tekrarına bakmak döngüyü her zaman yakalamaz.

Makinenin sonsuz döngüye girmesi gerçekten bir sorundur. Bu soruna durma problemi (halting problem) denir: Bir Turing makinesinin durup durmayacağını bilebilir miyiz?

> **Hocanın vurgusu:** Bir Turing makinesinin durup durmayacağını bilip bilemeyeceğimiz çok önemli bir sorudur.

Makine buradaki gibi çok basitse sonsuz döngüye girebileceğini söyleyebilirsiniz. Ya da bütün girişlerde durduğunu görebilirsiniz. Ama bu aslında oldukça zordur. Turing makinesi kolayca çok karmaşık hale gelebilir. Şimdiye kadar gördüğümüz örnekler çok kolaydı. Bir insanın görüp çözümleyemeyeceği kadar karmaşık bir makinede, durup durmayacağını bilemezsiniz. Hoca bunu felsefi bir soru ve onun cevabı olarak anlattı: "Size bir program versem, durup durmayacağını söyleyebilir misiniz?" Hocanın cevabı, program çok basit değilse bunu söyleyemeyeceğimizdir.

> **Ders sonrası not:** Hoca "çok basit olmayan bir programın durup durmayacağını söyleyemeyiz" dedi. Kesin sonuç bundan farklıdır: Durma problemi karar verilemezdir (undecidable). Yani her makine ve her giriş için doğru cevabı veren genel bir algoritma yoktur. Bu, karmaşık bir programın durduğunun hiçbir zaman gösterilemeyeceği anlamına gelmez.

20. Evde İzlenecek Örnek: $\{a^n b^n\}$ Makinesi

Süre bitmek üzereyken hoca, evde çalışılması için bir makine çizdi. Ayrıntılı anlatmadı, ama bu kadar basit bir makinenin bile kolayca karmaşıklaştığını göreceğinizi söyledi. Giriş $aabb$ ile deneyip nasıl çalıştığını kendiniz görmenizi istedi.

Durum	Okunan	Yazılan, yön	Sonraki
q_0	a	x, R	q_1
q_0	y	y, R	q_3
q_1	a	a, R	q_1
q_1	y	y, R	q_1
q_1	b	y, L	q_2
q_2	y	y, L	q_2
q_2	a	a, L	q_2
q_2	x	x, R	q_0
q_3	y	y, R	q_3
q_3	$\diamond$	$\diamond, L$	q_4 (kabul)

> **Ek bilgi (derste anlatılmadı):** Makinenin fikri şudur. En soldaki a'yı x yapar, sağa gidip ilk b'yi y yapar, sola dönüp x'e kadar gelir ve tekrarlar. a kalmayınca q_3'te yalnız y'lerden geçer. Ardından boşluk gelirse kabul eder.

Hocanın istediği $aabb$ girişi için hesaplamanın tamamı şöyledir (ara adımlar eklendi, Python simülasyonuyla üretildi):

Zaman	ID	Kullanılan geçiş
0	$q_0\,aabb$	$a \to x, R$
1	$x\,q_1\,abb$	$a \to a, R$
2	$xa\,q_1\,bb$	$b \to y, L$
3	$x\,q_2\,ayb$	$a \to a, L$
4	$q_2\,xayb$	$x \to x, R$
5	$x\,q_0\,ayb$	$a \to x, R$
6	$xx\,q_1\,yb$	$y \to y, R$
7	$xxy\,q_1\,b$	$b \to y, L$
8	$xx\,q_2\,yy$	$y \to y, L$
9	$x\,q_2\,xyy$	$x \to x, R$
10	$xx\,q_0\,yy$	$y \to y, R$
11	$xxy\,q_3\,y$	$y \to y, R$
12	$xxyy\,q_3\,\diamond$	$\diamond \to \diamond, L$

Zaman	ID	Kullanılan geçiş
13	$xxy\,q_4\,y$	Çıkan geçiş yok, durur ve kabul eder

Ders sonrası not: Hoca dili sözlü olarak "n birden büyük" diye okudu. Doğrusu $n \geq 1$'dir: $\{a^n b^n \mid n \geq 1\}$. Makine ab'yi kabul eder, boş dizgiyi kabul etmez. Uzunluğu en fazla 10 olan bütün $\{a, b\}$ dizgilerinde (2047 dizgi) Python ile tam tarandı ve kabul edilen dizgiler tam olarak bunlar çıktı. Hocanın kopyalarken hata yapmış olabileceği çekincesine de gerek kalmadı: Bu tablo doğru çalışır.

Dersin Toparlaması

Bu derste önce PDA'ları hatırladık, sonra her CFG'nin üç durumlu bir PDA'ya çevrilebildiğini gördük. Bu PDA'da iki geçiş hep aynıdır. Asıl iş q_1'deki döngüdedir: Her üretim kuralı için bir geçiş, her terminal için bir geçiş. Üretilen makine en soldan türetimi taklit eder. Yığında türetimin henüz eşleşmemiş kısmı durur. Ters yön olan PDA'dan CFG'ye dönüştürme atlandı. Böylece kitabın ikinci bölümü bitti.

Ardından dil hiyerarşisinde bir üst basamağa çıkıp Turing makinelerine başladık. Turing makinesi iki yönde sonsuz bir şerit üzerinde okur, yazar ve sola ya da sağa gider. Deterministiktir, ama geçiş fonksiyonu kısmi olabilir. Üç sonuç vardır: kabul durumunda durmak, kabul olmayan bir durumda durmak ve sonsuz döngüye girmek. Sonsuz döngü bizi durma problemine götürdü. Bölümün adındaki Church-Turing tezine ve deterministik olmayan Turing makinelerine sonra geleceğiz.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.